

Life OS

Engineering Handbook · Rev A

I · PRODUCT

- The thesis
- PR/FAQ — Life OS
- PR/FAQ — Today
- PR/FAQ — Body
- PR/FAQ — Activity
- PR/FAQ — Recovery
- PR/FAQ — The Coach
- Roadmap

II · ARCHITECTURE

- System context
- Module map
- The DailyMetrics spine
- Local-first topology
- Conflict resolution

III · INTEGRATIONS

- Apple HealthKit
- Whoop
- Supabase
- Foundation Models
- Screen Time

IV · REFERENCE

- DailyMetrics
- UserGoals
- Source records
- Goal evaluation
- MetricsStore

V · PRODUCTION

- Time & calendars
- HealthKit failures
- Whoop failures
- Sync failures
- UI & data display
- AI failures

VI · OPERATIONS

- Setup runbook
- Secrets
- Testing strategy
- Decision log





Life OS Engineering Handbook

A personal iOS app that unifies every signal from the devices you already wear and carry — Apple Watch, iPhone, Whoop, Renpho — and reasons over all of it to say what's working and what isn't.

REV A · 10 AUG 2026 IOS 26 · SWIFT 6 STATUS: V1 IN BUILD AUDIENCE: PERSONAL

This handbook is the single reference for how Life OS is built, why it is built that way, and what it does when things go wrong. It documents a system that is designed but not yet finished — V1 is in active implementation. Where the code and this document disagree, the code is right and this document is a bug.

Product

The thesis

Every health app you already have is a silo. Whoop knows your recovery but not that you skipped breakfast. Apple Health knows your steps but not that you blew the grocery budget and slept badly because of it. Each one gives advice that is correct in isolation and useless in context.

Life OS is built on a single bet: **the value is in the join, not the collection**. A coach that can see you missed meditation, logged 5 hours of sleep, journalled "low focus," and went 900 calories over can say something no single-purpose app can. That is the entire reason to build one app instead of using five.

Everything in the architecture follows from that bet. There is one table keyed by date that every source writes into and every reader reads from. Cheap manual modules ship early precisely *because* they make the coach smarter, not despite being simple.

Life OS

Life OS is a personal dashboard that pulls together everything your devices already know about you — weight from your scale, recovery from your strap, movement from your watch, and what you ate from a photo — and answers one question every morning: *am I on track?* Open it and a grid of dots shows the month at a glance. Filled means a good day. The app knows what a good day means because you told it.

Who is this for?

One person. This is a personal app, sideloaded on the author's own devices. There are no accounts, no other users, and no App Store distribution. That constraint removes an enormous amount of architecture and is load-bearing throughout this document.

Why not just use Apple Health?

Apple Health is an excellent store and a poor interpreter. It will show you a chart of anything and tell you the meaning of nothing. Life OS reads from it, then adds the layer Health deliberately doesn't: a definition of a good day, a streak that survives days you forgot to log, and eventually a coach that reads across domains.

What is explicitly *not* in this product?

Screen time analysis. It was requested and it cannot be built — see Screen Time for the technical reason. Rather than ship a hollow version, it is cut.

What's the hardest part?

Not the integrations. It's friction: any app that requires daily manual logging dies in about eleven days. Every product decision here is subordinate to keeping the daily cost near zero — which is why food logging is a photo, not a form.

Today

Today is the hub. It opens with the date at the size of a poster, a strip of the current week, and a grid of dots — one per day this month. Filled dots are days you hit your targets, hollow dots are days you didn't, and today is orange. Underneath, four tiles: steps, sleep, weight, recovery. You can read the whole month in about a second without processing a single number.

What makes a dot fill?

Meeting **3 of 4** daily goals: steps $\geq 8,000$, sleep $\geq 7h$, exercise ≥ 30 min, water $\geq 2,500$ ml. All five values — the four goals and the required count — are editable in Settings. The rule is deliberately explicit; a vague definition makes the entire grid meaningless.

What happens on a day with no data at all?

It renders as no-data, not as a miss, and it is **excluded from streak arithmetic entirely**. Leaving your watch on the charger is not a moral failure, and a system that punishes it teaches you to distrust the streak.

And a day with *some* data?

Judged on what it has. If steps are logged at 500 and nothing else was recorded, that's a miss. The alternative — treating partial days as blank — would let forgetting to log water silently launder a bad day into a neutral one.

If I change a goal, does history change?

Yes. Status is computed on read, never stored, so past dots re-evaluate against the new target immediately. This is intentional: the grid answers "how am I doing against my current standard," not "how did I do against a standard I've since abandoned."

Body

Body is a teal screen with your weight at 96 points and nothing competing with it. Below, the week's change with direction, and a fourteen-day bar chart where each bar is a weigh-in. Days you didn't step on the scale are visibly gaps, not interpolations.

Where does weight come from?

Your Renpho scale, via Apple Health. Renpho already syncs there, so Life OS never talks to Renpho directly — one less integration, one less thing to break, and no third-party credentials to store.

Why show a weekly delta instead of a trend line?

Daily bodyweight is mostly water and noise. A seven-day delta is the shortest window where the signal beats the noise, and it's the number that actually changes behaviour.

What if I weigh myself twice in one day?

The row for that day holds one weight; the most recent write wins. This is a deliberate simplification — for a daily-granularity app, the second weigh-in is the better one anyway.

Activity

Activity is amber, with today's step count as the hero and three tiles beneath it: active minutes, energy burned, and resting heart rate. It answers "did I move today" before you've finished unlocking the phone.

Both my iPhone and Watch count steps. Does that double?

No. Life OS uses `HKStatisticsCollectionQuery`, which de-duplicates overlapping samples across sources. Summing raw samples client-side — the obvious naive approach — would roughly double every step count on days you carried both devices.

Why is resting heart rate on the Activity screen and not Recovery?

Because it comes from HealthKit and is available in V1, while Recovery's other metrics depend on the Whoop integration landing. It moves to Recovery once that ships.

Recovery

Recovery is blue, and leads with Whoop's recovery percentage — the number you'd otherwise open the Whoop app to see — with HRV, day strain, and sleep performance beneath it.

Whoop syncs to Apple Health. Why build an OAuth integration at all?

Whoop writes sleep, workouts, and heart rate to Health, but **not Recovery % or Day Strain** — those are proprietary and exist only through Whoop's own API. They are also the two numbers most people wear the strap for. Hence the integration.

What does this screen show before Whoop is connected?

A designed empty state — an em dash and "Connect Whoop in Settings" — never `0%`. A zero recovery score is a medically alarming number to display to someone who simply hasn't logged in yet.

What if I didn't wear the strap last night?

Whoop returns no recovery record for that cycle, the field stays `nil`, and the screen shows the same empty state. No interpolation, no carry-forward from yesterday.

The Coach

Every morning, one card: what you did well yesterday, what slipped, and one thing to fix today. Built from your actual weight, sleep, recovery, steps, calories, and water. Ask it anything — "why is my recovery tanking?" — and it queries your real records to answer rather than guessing.

Does my health data go to a server?

For the coach, no. It runs on Apple's on-device Foundation Models — free, private, offline, no API key. Only food photos leave the device, because the on-device model is text-only and cannot see an image.

How does it avoid making numbers up?

Tool calling. The model doesn't receive your data in a prompt; it calls Swift functions that query `DailyMetrics` and returns what they hand back. A number it had to look up is a number it cannot hallucinate — and the context stays small enough for a 3B on-device model to handle.

What if the on-device model isn't available?

`SystemLanguageModel.availability` is checked at runtime. On a device without Apple Intelligence, or while the model is still downloading, the coach degrades to unavailable rather than silently shipping health data to a cloud provider without asking.

Roadmap

V1 is the only slice specified in detail. Each subsequent slice gets its own spec and plan.

SLICE	CONTENTS	WHY HERE
V1	Design system, local data layer, Supabase sync, HealthKit, Whoop, five screens	De-risks the hard integrations before adding breadth
V2	AI coach — daily brief + chat, on-device with tool calling	Needs the data layer to exist first
V3	Nutrition — photo → Claude vision → macros; water logging	The only source that needs a network model
V4	Habits + streaks, Goals + progress bars	Cheap by construction; makes the coach smarter
V5	Journal, Notes	Mood and focus are strong coach signals
V6	Money — manual income, expenses, net worth, savings rate	Manual entry; no bank integration
V7	Content planner calendar	Independent of everything else

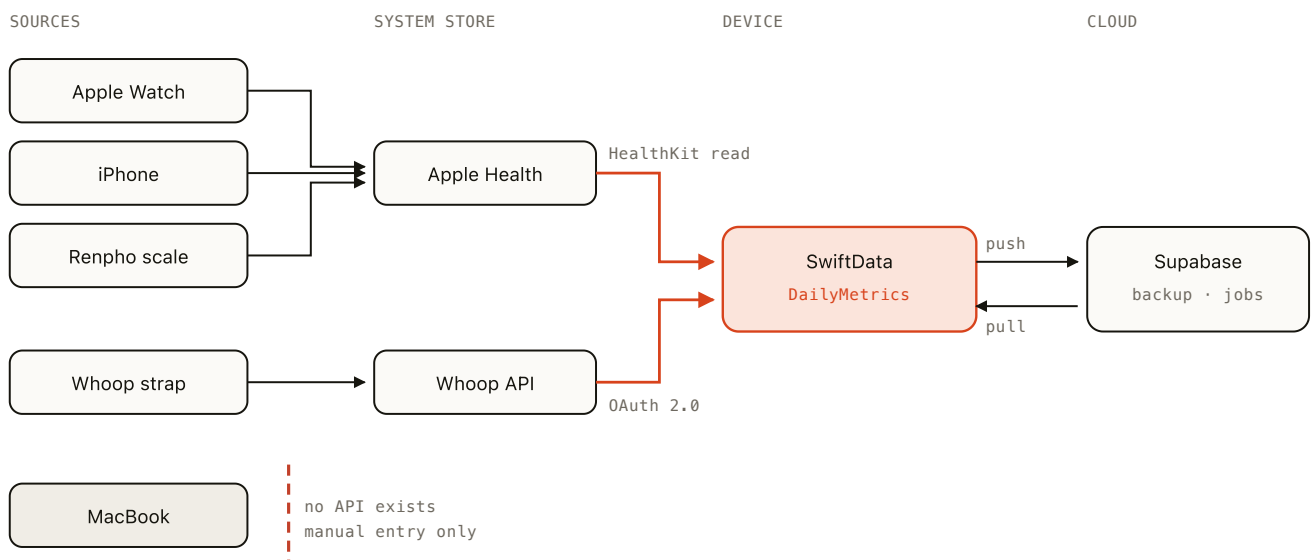
WHY THE CHEAP MODULES AREN'T AN AFTERTHOUGHT

V4–V7 are each one SwiftData model, one hue token, and one screen assembled from existing components. They are cheap *because* V1 builds the design system and the daily spine properly. They also carry disproportionate value: every domain added to the join makes the coach materially better at its job.

Architecture

System context

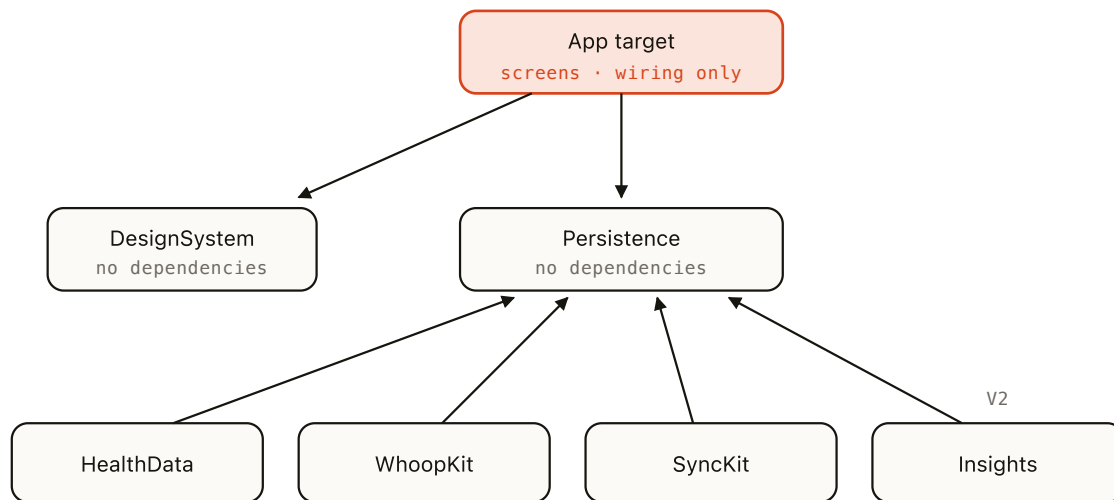
Four devices produce data, two of them reach the app through Apple Health, one through a REST API, and one — your MacBook — cannot reach it at all. Everything converges on a single local store before anything renders.



Every automatic source converges on one local table before anything renders. The MacBook has no programmatic path at any privilege level — that is a platform fact, not a missing feature.

Module map

`LifeOSKit` is a local Swift package providing compile-time boundaries. The app target contains only screens and wiring. The dependency rules are enforced by the compiler, not by convention.



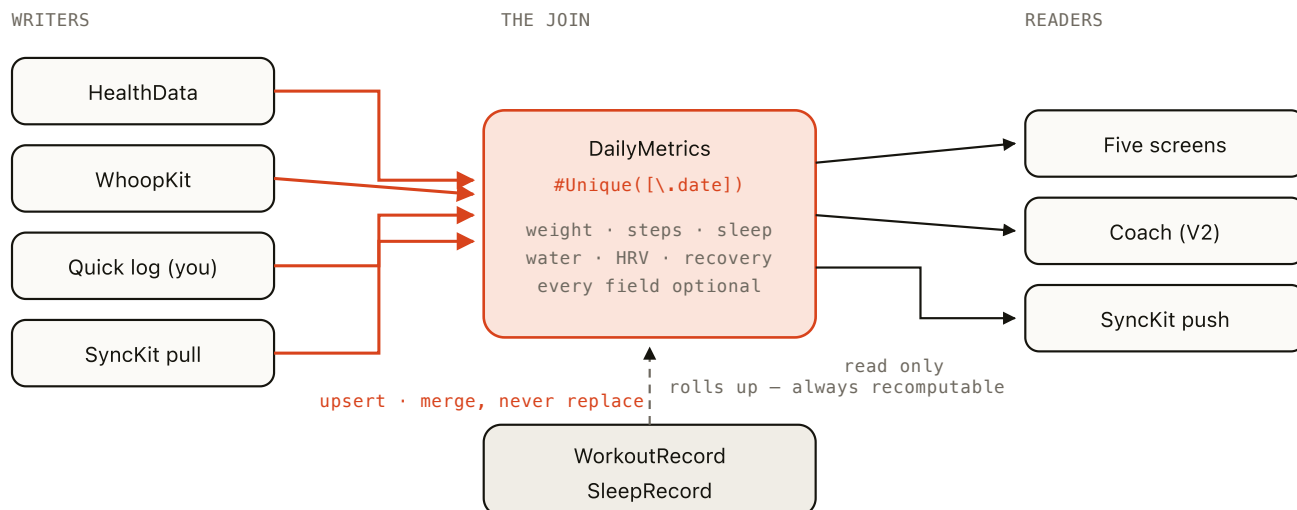
Arrows read "depends on". No arrow between siblings – that is the point.

The four integration modules are siblings that cannot see each other. Rewriting the Whoop client cannot touch a view, and `Insights` reads only from `Persistence` — it never calls HealthKit or Whoop directly.

RULE	ENFORCED HOW	WHY
Nothing outside <code>HealthData</code> imports HealthKit	Package dependency graph	HealthKit types never leak into views; the module is swappable and testable
Nothing outside <code>WhoopKit</code> knows Whoop exists	Package dependency graph	A vendor change is contained to one target
Nothing outside <code>SyncKit</code> imports the Supabase SDK	Package dependency graph	The app works identically with sync disabled
<code>Insights</code> reads <code>Persistence</code> only	Package dependency graph	The coach cannot make network calls mid-conversation
<code>DesignSystem</code> depends on nothing	Package dependency graph	Components stay previewable and reusable across every module

The DailyMetrics spine

This is the single most consequential decision in the system. **One row per calendar day.** Every source writes into it; every reader reads from it.



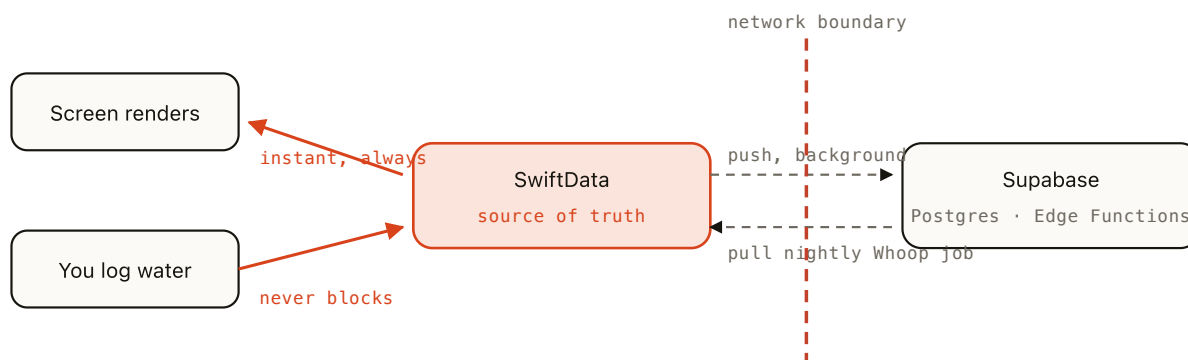
Sources merge into the row rather than replacing it, so HealthKit and Whoop writing the same day never clobber each other. `DailyMetrics` is derived state: it can always be rebuilt from the source records beneath it.

WHAT THE SINGLE-TABLE CHOICE BUYS

- **Trends become a sort**, not a multi-source asynchronous join.
- **The coach queries one table** instead of orchestrating two network APIs mid-conversation — which is what makes a 3B on-device model viable.
- **Offline works by default**, because nothing renders from a live network call.
- **Adding a source later** means writing into the same row, not building a new pipeline.

Local-first topology

SwiftData on-device is the source of truth for the UI. Supabase is backup, sync, and a place to run jobs that a sleeping phone cannot.



Everything left of the boundary works on a plane.
A sync failure surfaces in Settings as a stale timestamp — never as a block

Reads and writes both terminate locally. The dashed sync paths are the only network-dependent edges in the entire app, and neither is on a user-facing critical path.

Conflict resolution

Field-level last-write-wins, using `updatedAt` . Not row-level.

Row-level LWW would be a bug here. The nightly Whoop job writes recovery fields at 04:00; if you logged water at 23:00 the night before, a row-level overwrite silently discards it. Field-level merge means each writer touches only its own columns.

This is defensible only because there is exactly one human user and the writers rarely contest the same field: HealthKit owns the health columns, the Whoop job owns the Whoop columns, and you own the manual ones. If this app ever became multi-user, this strategy would need replacing outright.

KNOWN LIMITATION

LWW depends on clock agreement between the device and the Edge Function. Device clock skew of more than a few minutes can cause a stale write to win. Accepted for a single-user personal app; documented so it isn't rediscovered as a mystery bug.

Integrations

Apple HealthKit

The primary source. Read-only in V1 — the app requests no write permissions, which keeps the authorization surface minimal and means Life OS can never corrupt your Health data.

TYPES READ

METRIC	IDENTIFIER	QUERY
Weight	<code>bodyMass</code>	Anchored
Steps	<code>stepCount</code>	Statistics collection
Active energy	<code>activeEnergyBurned</code>	Statistics collection
Exercise minutes	<code>appleExerciseTime</code>	Statistics collection
Resting HR	<code>restingHeartRate</code>	Anchored
HRV	<code>heartRateVariabilitySDNN</code>	Anchored
Water	<code>dietaryWater</code>	Statistics collection
Sleep	<code>sleepAnalysis</code> (category)	Anchored
Workouts	<code>HKWorkoutType</code>	Anchored

FRESHNESS

- `HKAnchoredObjectQuery` with a **persisted anchor per type**, so each sync fetches only what changed since last time.
- `enableBackgroundDelivery` so iOS wakes the app when new samples land.
- `HKStatisticsCollectionQuery` for daily aggregates — this de-duplicates across sources. Summing raw samples yourself double-counts steps on any day you carried both an iPhone and a Watch.

THE AUTHORIZATION TRAP

HealthKit read authorization is **deliberately opaque**. The API will not tell you whether the user denied a type — only whether you have asked. Branching on read authorization status is therefore always a bug. Query anyway, and treat an empty result as "no data," rendering an explicit empty state. Apple designed it this way so apps cannot detect that you are hiding a condition from them.

Whoop

Base URL `https://api.prod.whoop.com`, API **v2**. v1 is deprecated. Note that v2 identifies sleep by UUID rather than integer ID, and recovery webhooks reference the associated *sleep* UUID, not the cycle ID — a migration footgun worth knowing before you read the payloads.

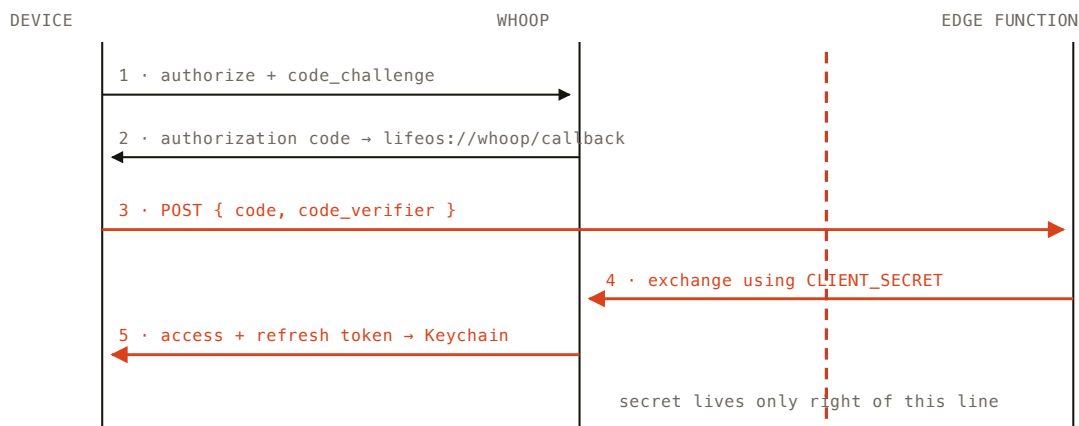
SCOPES

```
offline
read:cycles
read:sleep
read:recovery
read:workout
read:body_measurement
read:profile
```

`offline` is required to receive a refresh token. Without it, the integration breaks the first time the access token expires.

OAuth — AND WHY THE SECRET NEVER SHIPS

Whoop's token exchange requires a client secret. Embedding it in the app binary would make it extractable with `strings`. Supabase Edge Functions solve this cleanly: the app does PKCE and obtains a *code*; the function exchanges it.



Steps 3–5 are the whole point: the client secret is used at step 4, on the server, and never crosses back onto the device. Without this, the integration would be unshippable to anyone but its author.

NIGHTLY JOB AND WEBHOOKS

- A scheduled Edge Function (`pg_cron`) pulls the previous day's cycle, recovery, and sleep overnight, so morning data exists before you open the app.
- A `whoop-webhook` function receives push updates. Signature verified against `WHOOB_WEBHOOK_SECRET` ; unverified requests are rejected outright.
- Webhooks are only possible because a public HTTPS endpoint now exists. A phone alone cannot receive them.

Supabase

SCHEMA

Mirrors the local SwiftData models: `daily_metrics` , `workout_records` , `sleep_records` , `whoop_raw` , and `sync_state` .

`whoop_raw` stores unprocessed API payloads. This is deliberate: when Whoop's response shape differs from the documentation — and it will — you can re-derive every downstream value without re-fetching a month of history against a rate limit.

ROW LEVEL SECURITY

RLS IS ENABLED ON EVERY TABLE, WITHOUT EXCEPTION

Policies scope all access to `auth.uid()` . This is precisely what makes shipping the anon key inside the app safe. An RLS misconfiguration does not throw — it returns an *empty result set*, which looks exactly like "no data yet." Verify policies with a real query before trusting an empty screen.

EDGE FUNCTIONS

FUNCTION	PURPOSE	SECRETS USED
whoop-token-exchange	Exchange auth code / refresh token	WHOOOP_CLIENT_SECRET
whoop-sync	Scheduled nightly pull	WHOOOP_CLIENT_SECRET , SERVICE_ROLE_KEY
whoop-webhook	Receive Whoop push events	WHOOOP_WEBHOOK_SECRET , SERVICE_ROLE_KEY
analyze-meal v3	Food photo → macros via Claude vision	ANTHROPIC_API_KEY

Foundation Models

The coach runs on Apple's on-device model by default. All three tiers sit behind the same `LanguageModelSession` API, so routing is configuration rather than a rewrite.

TIER	USED FOR	CONTEXT	COST	AVAILABLE
SystemLanguageModel	Daily brief, chat, trend commentary	Small	Free, private, offline	iOS 26
PrivateCloudComputeLanguageModel	Monthly / quarterly deep analysis	32K	Free, private	iOS 27
Claude API	Food photo → macros (needs vision)	Large	~cents per photo	Any

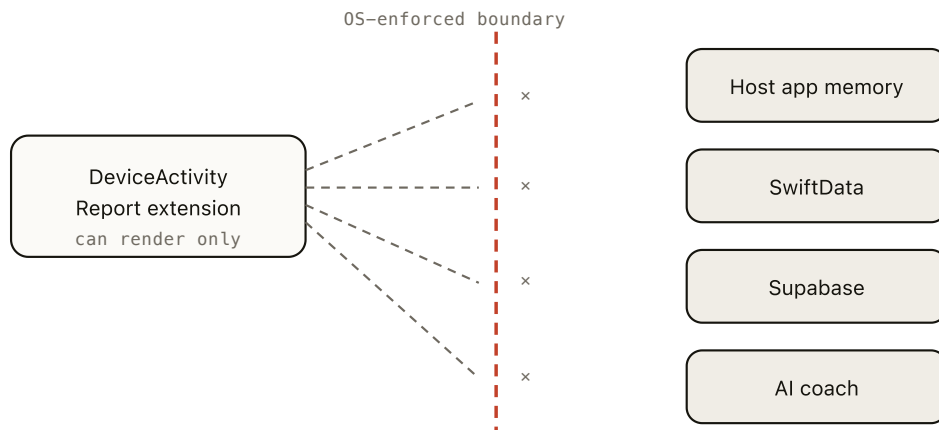
iOS 27 additionally introduces public `LanguageModel` and `LanguageModelExecutor` protocols, letting third-party providers implement the same interface. **Do not hand-roll a provider abstraction** — Apple ships one, and a bespoke layer would have to be torn out.

TOOL CALLING IS THE ANTI-HALLUCINATION MECHANISM

The model is not handed your data in a prompt. It calls Swift functions that query `DailyMetrics`. Two consequences: a number it had to look up is a number it cannot invent, and the context stays small enough for an on-device model to handle a month of history.

Screen Time — why it is cut

Screen time was an explicit product requirement. It cannot be built as specified, and the reason is worth documenting precisely so it is not re-attempted.



Usage figures can be displayed inside the report extension and nowhere else. The host app cannot read them into memory, persist them, transmit them, or pass them to the coach.

- **iPhone / iPad:** a screen-time *view* is possible; screen-time *analysis alongside health data* is not. It would also require Apple to grant the Family Controls distribution entitlement.
- **MacBook:** no API exists at any privilege level. Manual entry only, if ever.

Since the entire product thesis is the join across domains, a signal that provably cannot enter the join has no home here. Cut rather than half-built.

Reference

DailyMetrics

One row per calendar day. The join key for the entire application.

PROPERTY	TYPE	WRITTEN BY	NOTES
<code>date</code>	<code>Date</code>	—	Always <code>startOfDay</code> . Unique. Never a timestamp.
<code>weightKg</code>	<code>Double?</code>	HealthKit, Quick log	Renpho → Health → here
<code>steps</code>	<code>Int?</code>	HealthKit	De-duplicated across sources
<code>activeEnergyKcal</code>	<code>Double?</code>	HealthKit	
<code>exerciseMinutes</code>	<code>Int?</code>	HealthKit	
<code>sleepMinutes</code>	<code>Int?</code>	HealthKit	Attributed to the wake day
<code>waterML</code>	<code>Double?</code>	Quick log	No automatic source exists
<code>restingHR</code>	<code>Double?</code>	HealthKit	
<code>hrvMs</code>	<code>Double?</code>	HealthKit	SDNN
<code>whoopRecoveryPct</code>	<code>Double?</code>	WhoopKit	Exists nowhere else
<code>whoopDayStrain</code>	<code>Double?</code>	WhoopKit	Exists nowhere else
<code>whoopSleepPerformancePct</code>	<code>Double?</code>	WhoopKit	
<code>updatedAt</code>	<code>Date</code>	MetricsStore	Drives field-level LWW
<code>syncedAt</code>	<code>Date?</code>	SyncKit	<code>nil</code> = never synced

EVERY METRIC IS OPTIONAL, AND THAT IS NOT NEGOTIABLE

A missing value and a zero must never be representable as the same thing. Zero steps means you didn't move; `nil` steps means the watch was on the charger. In a health app a false zero is worse than a blank — it is a wrong number presented with total confidence.

UserGoals

Singleton by convention — exactly one row, created with defaults on first access.

PROPERTY	TYPE	DEFAULT	RANGE IN SETTINGS
stepsGoal	Int	8,000	1,000 – 30,000
sleepMinutesGoal	Int	420 (7h)	240 – 660
exerciseMinutesGoal	Int	30	5 – 180
waterMLGoal	Double	2,500	500 – 6,000
requiredCount	Int	3	1 – 4

Source records

`WorkoutRecord` and `SleepRecord` hold data that is genuinely not daily. They roll *up* into `DailyMetrics`, which means the daily row is always safe to recompute and never the only copy of anything.

Goal evaluation

```
public func evaluate(_ reading: DayReading,
                    against targets: GoalTargets) -> DayStatus
```

Returns `.onTarget`, `.missed`, or `.noData`.

INPUT	RESULT	RATIONALE
No metrics present at all	<code>.noData</code>	Not a failure; excluded from streaks
$\geq$ <code>requiredCount</code> goals met	<code>.onTarget</code>	Default 3 of 4
Some data, below required count	<code>.missed</code>	An unlogged metric cannot launder a bad day
Exactly at a goal value	Counts as met	<code>>=</code> , not <code>></code>

```
public func currentStreak(statuses: [DayStatus]) -> Int
```

Expects most-recent-first ordering. `.noData` days are transparent — they neither extend nor break a streak. `.missed` terminates immediately.

MetricsStore

The only path by which anything writes to `DailyMetrics`. It enforces the one-row-per-day invariant so callers cannot get it wrong.

```
@discardableResult
func upsert(date: Date, apply: (DailyMetrics) -> Void) throws -> DailyMetrics
func metrics(from: Date, to: Date) throws -> [DailyMetrics]    // most-recent-first
func goals() throws -> UserGoals                               // creates defaults on fir
```

`upsert` normalises any timestamp to `startOfDay` and **merges** — fields the closure doesn't touch are preserved. That is what lets HealthKit and Whoop write the same row safely.

Production edge cases

The register below is the difference between a demo and something you can rely on. Each row is a condition that will occur in normal use, not a theoretical one.

Time and calendars

CONDITION	BEHAVIOUR	STATUS
Sleep spans midnight	Attributed to the <i>wake</i> day via <code>SleepRecord.attributedDate</code>	Designed
Travel across time zones	"Today" follows the device's current zone; a day can be shortened or lengthened	Accepted
DST transition (23h or 25h day)	<code>Calendar.startOfDay</code> handles it; never compute days by adding 86,400 seconds	Designed
Month grid at month boundaries	Leading blanks computed from <code>firstWeekday</code> ; padded to whole rows	Tested
Data arrives for a past day	<code>upsert</code> merges into the existing row; dot grid re-evaluates on next read	Designed
Future-dated day	Rendered <code>.future</code> by position, never evaluated against goals	Tested
Device clock skew vs server	Can cause a stale LWW win	Known limitation

HealthKit failures

CONDITION	BEHAVIOUR
User denies read access	Undetectable by design. Query anyway; empty result renders an empty state, and Settings links to Health's permission screen
Permission revoked mid-use	Same path — data simply stops arriving; the last-synced timestamp in Settings goes stale
iPhone and Watch both counting steps	De-duplicated by <code>HKStatisticsCollectionQuery</code> . Never sum raw samples
Background delivery doesn't fire	iOS throttles aggressively. Also sync on foreground; never depend solely on background wake
Anchor invalidated (restore from backup)	Anchor reset triggers a full re-read; <code>upsert</code> merge makes this idempotent
Weight recorded in pounds	Read through <code>HKUnit</code> conversion, stored canonically in kg
Watch not worn all day	Fields stay <code>nil</code> . Day is <code>.noData</code> , streak survives
Simulator has no health data	Expected. Seeded data covers development; integration is device-only

Whoop failures

CONDITION	BEHAVIOUR
Access token expires mid-request	Refresh via Edge Function, retry once, then surface reconnect
Refresh token rotated	Whoop rotates on use — persist the <i>new</i> refresh token atomically. Reusing an old one invalidates the whole chain
Refresh fails with auth error	Clear Keychain, show "reconnect Whoop" in Settings. No silent retry loop
Rate limited (429)	Respect <code>Retry-After</code> ; exponential backoff in the nightly job
Webhook delivered twice	Idempotent by record UUID — reprocessing is a no-op
Webhook signature invalid	Rejected before parsing. Never trust an unverified payload
Strap not worn	No recovery record; fields stay <code>nil</code> ; screen shows its empty state
Whoop cycle boundary $\neq$ calendar day	Cycles are attributed to the day the cycle <i>starts</i> ; documented so the mismatch isn't read as a bug
Payload shape differs from docs	Raw payloads retained in <code>whoop_raw</code> ; re-derive without re-fetching
User revokes access in Whoop's app	Next refresh 401s → same reconnect path

Sync failures

CONDITION	BEHAVIOUR
No network	Every screen reads local. Failure surfaces only as a stale timestamp in Settings
Write while offline	Committed locally, <code>syncedAt</code> stays <code>nil</code> , pushed on next successful sync
Partial batch failure	Per-row <code>syncedAt</code> means only unsynced rows retry. No all-or-nothing batches
Same row edited locally and by the nightly job	Field-level LWW — the job's Whoop columns and your water column both survive
Supabase session expires	Refresh silently; on hard failure, sync pauses and Settings shows it. The app keeps working
RLS policy misconfigured	Returns empty, does not throw. Indistinguishable from "no data" — verify policies explicitly
Schema drift SwiftData ↔ Postgres	Decoding failures logged per-row and skipped; one bad row never halts a sync

UI and data display

CONDITION	BEHAVIOUR
First launch, no data	Seeded 60 days in development; in production every hero shows its designed empty state
Metric missing	Em dash at 35% opacity — never <code>0</code>
Goal changed retroactively	All history re-evaluates. Intentional: the grid measures against your current standard
Streak across a no-data gap	Gap is transparent; streak continues
Progress bar with goal of zero	Ratio clamped to 0...1; goal ranges in Settings exclude zero
Weight delta with < 7 days history	Delta suppressed entirely rather than computed from a short window
Very large numbers	<code>minimumScaleFactor(0.5)</code> and <code>lineLimit(1)</code> on hero numerals
Dark mode	Every hue has a dark variant; neutral tokens resolve per scheme. No color defined only inside a scheme branch
Dynamic Type at accessibility sizes	Hero numerals scale down; tiles reflow. Verify at XXL before shipping

AI failures v2

CONDITION	BEHAVIOUR
Device lacks Apple Intelligence	<code>SystemLanguageModel.availability</code> checked; coach degrades to unavailable rather than silently uploading health data
Model still downloading	Reported as unavailable with the reason surfaced, not a spinner
Context window exceeded	Tool calling keeps context small by construction; deep analysis routes to the 32K PCC tier
Guardrail refusal	Surfaced plainly. Never retried with a reworded prompt to evade it
Tool returns no rows	Model is told "no data for that range" explicitly, so it says so instead of inventing a trend
User asks about a future date	Tool returns empty; no speculation

Operations

Setup runbook

1 · LOCAL BUILD

```
git clone https://github.com/shivvyas2/LifeOS.git
cd LifeOS
cp Config/Secrets.example.xcconfig Config/Secrets.xcconfig
swift test --package-path LifeOSKit      # no simulator needed
open LifeOS.xcodeproj
```

In Xcode: Swift Language Version → 6, Minimum Deployments → iOS 26.0, and add `LifeOSKit` via *Add Package Dependencies* → *Add Local*.

2 · SUPABASE

```
supabase init
supabase link --project-ref <ref>
supabase db push                # schema + RLS policies
supabase secrets set WHOOP_CLIENT_SECRET=... \
                             WHOOP_WEBHOOK_SECRET=...
supabase functions deploy whoop-token-exchange
supabase functions deploy whoop-sync
supabase functions deploy whoop-webhook
```

Then put the project URL and **anon** key into `Config/Secrets.xcconfig`.

3 · WHOOP DEVELOPER APP

1. Register at `developer.whoop.com` (requires an active Whoop membership).
2. Redirect URI: `lifeos://whoop/callback`
3. Request all seven scopes listed in Whoop, including `offline`.
4. Client ID → `Config/Secrets.xcconfig`. Client secret → `supabase secrets set`, **never the xcconfig**.
5. Webhook URL → the deployed `whoop-webhook` function URL.

4 · DEVICE

HealthKit background delivery and the Whoop OAuth round trip cannot be verified in the simulator. Both require a physical device with a signing team configured.

Secrets

There is no runtime `.env` on iOS. The app is a signed bundle; anything shipped inside it is extractable. The governing rule is therefore: **what must never leak, must live server-side.**

TIER	LOCATION	CONTENTS	IN GIT?
1 · Build config	<code>Config/Secrets.xcconfig</code>	Supabase URL, anon key, Whoop client ID	Gitignored. Template committed
2 · Runtime	Keychain	Whoop access + refresh tokens, Supabase session	Never
3 · Server	Edge Function env	<code>WHOOO_CLIENT_SECRET</code> , <code>WHOOO_WEBHOOK_SECRET</code> , <code>ANTHROPIC_API_KEY</code> , <code>SERVICE_ROLE_KEY</code>	Never. Never on device either

Nothing in Tier 1 is truly secret. The Supabase anon key is *designed* to ship in clients — it is safe precisely because RLS gates every row — and the Whoop client ID is public by OAuth design. Tier 1 is gitignored for hygiene, not because a leak would be catastrophic.

SERVICE_ROLE IS THE ONE THAT MATTERS

It bypasses RLS entirely. It belongs only in an Edge Function environment — never in an xcconfig, never in the app target, never in git. Every other secret in this system is recoverable by rotation; this one grants total read and write access to everything.

Testing strategy

Logic that can be tested without a simulator lives in the package and is developed test-first. SwiftUI view bodies are verified by previews and a build — asserting on view structure is ceremony that breaks on every layout change without catching real defects.

TARGET	WHAT IS TESTED	HOW
Persistence	Goal evaluation, streak arithmetic, upsert invariants, rollup derivation	Swift Testing, in-memory ModelContainer
DesignSystem	Month grid calendar maths, hue completeness	Swift Testing, fixed UTC calendar
HealthData	Aggregation and anchor handling	Protocol-wrapped store, fixtures
WhoopKit	v2 payload decoding, token refresh state machine	Recorded payload fixtures
SyncKit	Field-level conflict resolution	Constructed clock skew
Views	Both themes, every hue, empty states	SwiftUI previews
Integrations	Background delivery, OAuth round trip	Manual, physical device

The package declares a macOS platform alongside iOS purely so `swift test --package-path Life05Kit` runs from the terminal without booting a simulator. Integration modules guard platform-specific imports with `#if canImport(HealthKit)` to preserve this.

Decision log

DECISION	CHOICE	REASON
Audience	Personal only	Removes accounts, App Store review, multi-user architecture
Weight source	HealthKit via Renpho's sync	No Renpho API integration needed at all
Whoop	Direct API in V1	Recovery % and Strain exist nowhere else
Whoop client secret	Edge Function only	Would otherwise be extractable from the binary
Food logging	Photo → AI estimate	Lowest friction; friction is what kills these apps
AI coach	On-device Foundation Models, tool calling	Free, private, offline; can't hallucinate looked-up numbers
Provider abstraction	Use Apple's, don't build one	iOS 27 ships <code>LanguageModel</code> protocols
Data topology	Local-first, Supabase syncs	A health app that waits on the network is one you stop opening
Conflict resolution	Field-level LWW	Row-level would let the nightly job discard your manual entries
Screen time	Cut	Apple's API forbids reading the values into the app
Every metric optional	Yes	A false zero is worse than a blank in a health app
Dot grid rule	3 of 4, configurable	A vague rule makes the whole grid meaningless
No-data days	Excluded from streaks	Punishing a charging night teaches distrust of the streak
V1 scope	Foundation + health	De-risks the hard integrations before adding breadth

Life OS Engineering Handbook · Rev A · 10 August 2026 · Source of record:
docs/superpowers/specs/2026-08-10-life-os-design.md